

# Приложение

## S.1. Битовые множества

Как было сказано в разделе 12.5, битовые множества имитируют массивы битов булевых значений фиксированного размера и полезны для управления флагами. В старых программах на языках C и C++ для манипулирования массивами битов с помощью побитовых операций, таких как `&`, `|`, и `~`, использовался тип `long`. Преимущество класса `bitset` заключается в том, что битовые множества могут содержать произвольное количество битов и допускают применение дополнительных операций. Например, можно присвоить один бит, а также читать и записывать битовые множества в виде последовательности нулей и единиц.

Изменять количество битов в битовом множестве нельзя, так как оно является шаблонным параметром. Если требуется контейнер для переменного количества битов или булевых значений, можно использовать класс `vector<bool>` (см. раздел 7.3.6).

Класс `bitset` определен в заголовочном файле `<bitset>`.

```
#include <bitset>
```

В заголовочном файле `<bitset>` класс `bitset` определен как шаблонный класс, у которого количество битов является шаблонным параметром.

```
namespace std {  
    template <size_t Bits>  
        class bitset;  
}
```

В данном случае шаблонный параметр является не типом, а целочисленным значением без знака (см. раздел 3.2).

Шаблоны с разными шаблонными параметрами представляют собой разные типы. Сравнивать и комбинировать можно только битовые множества с одинаковым количеством битов.

### Новшества стандарта C++11

В стандарте C++98 были перечислены практически все свойства битовых множеств. Ниже приводится список наиболее важных свойств, добавленных в стандарте C++11.

- Битовые множества можно инициализировать строковыми литералами (см. раздел 12.5.1).
- Преобразования битовых множеств в числовые значения, и наоборот, поддерживают тип `unsigned long long`. Для этого была введена функция `to_ullong()` (см. раздел 12.5.1).
- Преобразования битовых множеств в строки, и наоборот, позволяют указывать символ, интерпретируемый как установленный или сброшенный бит.

- Функция-член `all()` позволяет проверять, установлены ли биты.
- Для использования битовых множеств в неупорядоченных контейнерах предусмотрена хеш-функция по умолчанию (см. раздел 7.9.2).

## S.1.1. Примеры использования битовых множеств

### Использование битовых множеств как наборов флагов

Первый пример демонстрирует использование битовых множеств для манипулирования наборами флагов. Каждый флаг имеет значение, определенное как тип перечисления. Значение типа перечисления используется как позиция бита в битовом множестве. В частности, биты представляют цвета. Таким образом, каждое значение перечисления определяет один цвет. Используя битовые множества, можно управлять переменными, содержащими любое сочетание цветов.

```
// contadapt/bitset1.cpp

#include <bitset>
#include <iostream>
using namespace std;

int main()
{
    // тип перечисления для битов
    // - каждый бит представляет цвет
    enum Color { red, yellow, green, blue, white, black, ...,
                numColors };

    // создаем битовое множество для всех битов/цветов
    bitset<numColors> usedColors;

    // задаем биты для двух цветов
    usedColors.set(red);
    usedColors.set(blue);

    // выводим данные о битовом множестве
    cout << "bitfield of used colors:  " << usedColors << endl;
    cout << "number of used colors:    " << usedColors.count() << endl;
    cout << "bitfield of unused colors: " << ~usedColors << endl;

    // если используется любой цвет
    if (usedColors.any()) {
        // цикл по всем цветам
        for (int c = 0; c < numColors; ++c) {
            // если используется текущий цвет
            if (usedColors[(Color)c]) {
                ...
            }
        }
    }
}
```

## Использование битовых множеств для ввода и вывода бинарных представлений

Полезным свойством битовых множеств является способность преобразовывать целочисленные значения в последовательность битов, и наоборот. Эту операцию легко выполнить с помощью временного битового множества.

```
// contadapt/bitset2.cpp

#include <bitset>
#include <iostream>
#include <string>
#include <limits>
using namespace std;

int main()
{
    // выводим несколько чисел в бинарном представлении
    cout << "267 as binary short:      "
          << bitset<numeric_limits<unsigned short>::digits>(267)
          << endl;
    cout << "267 as binary long:       "
          << bitset<numeric_limits<unsigned long>::digits>(267)
          << endl;
    cout << "10 000 000 with 24 bits:  "
          << bitset<24>(1e7) << endl;

    // записываем бинарное представление в строку
    string s = bitset<42>(12345678).to_string();
    cout << "12 345 678 with 42 bits: " << s << endl;

    // преобразуем бинарное представление в целочисленное значение
    cout << "\"1000101011\" as number:  "
          << bitset<100>("1000101011").to_ullong() << endl;
}
```

В зависимости от количества битов, используемых для представления типов `short` и `long long`, программа может выводить следующий результат:

```
267 as binary short:      0000000100001011
267 as binary long:       00000000000000000000000100001011
10 000 000 with 24 bits:  100110001001011010000000
12 345 678 with 42 bits:  0000000000000000000101111000110000101001110
"1000101011" as number:  555
```

В этом примере следующее выражение преобразовывает целое число 267 в битовое множество с количеством битов, которое используется для представления типа `unsigned short` (см. раздел 5.3).

```
bitset<numeric_limits<unsigned short>::digits>(267)
```

Оператор вывода для типа `bitset` выводит биты как последовательность символов 0 и 1.

Кроме того, битовое множество можно вывести сразу или использовать его как строку.

```
string s = bitset<42>(12345678).to_string();
```

Отметим, что до принятия стандарта C++11 для этого необходимо было писать

```
string s = bitset<42>(12345678).to_string<char, char_traits<char>,  
                               allocator<char> >();
```

потому что функция `to_string()` является членом шаблонного класса, а в стандарте не были определены значения по умолчанию для шаблонных параметров.

Аналогично следующее выражение преобразовывает последовательность бинарных символов в битовое множество, которое функция `to_ullong()` превращает в целочисленное значение:

```
bitset<100>("1000101011")
```

Отметим, что количество битов в битовом множестве должно быть меньше `sizeof(unsigned long long)`. Причина заключается в том, что если значение невозможно представить типом `unsigned long long`, то генерируется исключение<sup>1</sup>.

Кроме того, до принятия стандарта C++11 необходимо было явно преобразовать начальное значение в тип `string`.

```
bitset<100>(string("1000101011"))
```

## S.1.2. Подробное описание класса `bitset<>`

Класс `bitset` предусматривает следующие операции.

### Создание, копирование и уничтожение

Для битовых множеств определено несколько конструкторов. Однако для них не предусмотрены специальные копирующие конструкторы, операции присваивания и деструктора. Таким образом, битовые множества присваиваются и копируются с помощью стандартной операции побитового копирования.

**`bitset<bits>::bitset ()`**

- Конструктор по умолчанию.
- Создает битовое множество, в котором все биты инициализированы нулем.
- Например:

```
bitset<50> flags; // флаги: 0000...000000  
// таким образом, имеем 50 сброшенных битов  
bitset<bits>::bitset (unsigned long long value)
```

**`bitset<bits>::bitset (unsigned long long value)`**

- Создает битовое множество, инициализированное битами целочисленного значения *value*.

---

<sup>1</sup> До принятия стандарта C++11 тип `unsigned long` не поддерживался, поэтому в этом месте можно было вызвать только функцию `to_ulong()`. Функцию `to_ulong()` все еще можно вызывать, если количество битов меньше, чем `sizeof(unsigned long)`.



- Если количество битов в значении *value* слишком мало, старшие биты инициализируются нулем.
- До принятия стандарта C++11 значение *value* имело тип `unsigned long`.
- Например:

```
bitset<50> flags(7); // флаги 0000...000111
explicit bitset<bits>::bitset (const string& str)
bitset<bits>::bitset (const string& str, string::size_type str_idx)
bitset<bits>::bitset (const string& str,
                    string::size_type str_idx,
                    string::size_type str_num)
bitset<bits>::bitset (const string& str,
                    string::size_type str_idx,
                    string::size_type str_num,
                    string::charT zero)
bitset<bits>::bitset (const string& str,
                    string::size_type str_idx,
                    string::size_type str_num,
                    string::charT zero, string::charT one)
```

- Все версии создают битовое множество, инициализированное строкой *str* или подстрокой строки *str*.
- Строка или подстрока может содержать только символы '0' и '1' (или *zero* и *one*).
- *str\_idx* — это индекс первого символа строки *str*, используемый для инициализации.
- Если параметр *str\_num* пропущен, то используются все символы от *str\_idx* до конца строки *str*.
- Если строка или подстрока имеет меньше символов, чем требуется, то старшие биты инициализируются нулями.
- Если строка или подстрока имеет больше символов, чем требуется, то остальные символы игнорируются.
- Если *str\_idx* > *str.size()*, то генерируется исключение `out_of_range`.
- Если один из символов не равен ни '0'/*zero*, ни '1'/*one*, то генерируется исключение `invalid_argument`.
- Параметры *zero* и *one* появились в стандарте C++11.
- Отметим, что этот конструктор является шаблонным членом класса (см. раздел 3.2). По этой причине не предусмотрено неявное преобразование типа `const char*` в `string` для первого параметра, который до принятия стандарта C++11 должен был быть строковым литералом.
- Например:

```
bitset<50> flags(string("1010101")); // флаги: 0000...0001010101
bitset<50> flags(string("1111000"),2,3); // флаги: 0000...0000000110

explicit bitset<bits>::bitset (const charT* str)
bitset<bits>::bitset (const charT* str, string::size_type str_num)
bitset<bits>::bitset (const charT* str, string::size_type str_num,
                    string::charT zero)
```

```
bitset<bits>::bitset (const charT* str, string::size_type str_num,
                      string::charT zero, string::charT one)
```

- Все версии создают битовое множество, инициализированное строкой символов *str*.
- Строка или подстрока может содержать символы '0' и '1' (или *zero* и *one*).
- Если параметр *str\_num* пропущен, то используются все символы строки *str*.
- Если строка *str* содержит меньше символов, чем требуется (параметр *str\_num* выходит за пределы допустимого диапазона), то старшие биты инициализируются нулем.
- Если параметр *str* содержит больше параметров, чем требуется, то остальные символы игнорируются.
- Если один из символов не равен ни '0'/*zero*, ни '1'/*one*, то генерируется исключение *invalid\_argument*.
- Параметры *zero* и *one* появились в стандарте C++11.
- Например:

```
bitset<50> flags("1010101"); // флаги: 0000...0001010101
```

## Немодифицирующие операции

```
size_t bitset<bits>::size () const
```

- Возвращает количество битов (т.е. значение *bits*).

```
size_t bitset<bits>::count () const
```

- Возвращает количество установленных битов (т.е. битов со значением 1).

```
bool bitset<bits>::all () const
```

- Возвращает признак того, что все биты установлены.
- Начиная со стандарта C++11.

```
bool bitset<bits>::any () const
```

- Возвращает признак того, что хотя бы один бит установлен.

```
bool bitset<bits>::none () const
```

- Возвращает признак того, что ни один бит не установлен.

```
bool bitset<bits>::test (size_t idx) const
```

- Возвращает признак того, что бит в позиции *idx* установлен.
- Если *idx* >= *size()*, генерирует исключение *out\_of\_range*.

```
bool bitset<bits>::operator == (const bitset<bits>& bits) const
```

- Возвращает признак того, что все биты объекта *\*this* и параметра *bits* имеют одно и то же значение.

```
bool bitset<bits>::operator != (const bitset<bits>& bits) const
```

- Возвращает признак того, что все биты объекта *\*this* и параметра *bits* имеют разные значения.

## Манипулирующие операции

**bitset<bits>& bitset<bits>::set ()**

- Устанавливает все биты равными true.
- Возвращает модифицированное битовое множество.

**bitset<bits>& bitset<bits>::set (size\_t idx)**

- Устанавливает бит в позиции *idx* равным true.
- Возвращает модифицированное битовое множество.
- Если *idx* >= *size()*, генерирует исключение *out\_of\_range*.

**bitset<bits>& bitset<bits>::set (size\_t idx, bool value)**

- Устанавливает бит в позиции *idx* в соответствии со значением *value*.
- Возвращает модифицированное битовое множество.
- Если *idx* >= *size()*, генерирует исключение *out\_of\_range*.
- До принятия стандарта C++11 параметр *value* имел тип *int*, так что значение 0 устанавливало бит равным false, а любое другое значение соответствовало значению true.

**bitset<bits>& bitset<bits>::reset ()**

- Сбрасывает все биты в значение false (присваивает 0 всем битам).
- Возвращает модифицированное битовое множество.

**bitset<bits>& bitset<bits>::reset (size\_t idx)**

- Сбрасывает бит в позиции *idx* в значение false.
- Возвращает модифицированное битовое множество.
- Если *idx* >= *size()*, генерирует исключение *out\_of\_range*.

**bitset<bits>& bitset<bits>::flip ()**

- Переключает все биты (устанавливает сброшенные биты, и наоборот).
- Возвращает модифицированное битовое множество.

**bitset<bits>& bitset<bits>::flip (size\_t idx)**

- Переключает бит в позиции *idx*.
- Возвращает модифицированное битовое множество.
- Если *idx* >= *size()*, генерирует исключение *out\_of\_range*.

**bitset<bits>& bitset<bits>::operator ^= (const bitset<bits>& bits)**

- Побитовая операция исключающего ИЛИ.

- Переключает все биты, установленные в параметре *bits*, оставляя все остальные биты неизменными.
- Возвращает модифицированное битовое множество.

**bitset<bits>& bitset<bits>::operator |=** (const bitset<bits>& *bits*)

- Побитовая операция ИЛИ.
- Устанавливает все биты, установленные в параметре *bits*, оставляя все остальные биты неизменными.
- Возвращает модифицированное битовое множество.

**bitset<bits>& bitset<bits>::operator &=** (const bitset<bits>& *bits*)

- Побитовая операция И.
- Сбрасывает все биты, не установленные в параметре *bits*, оставляя все остальные биты неизменными.
- Возвращает модифицированное битовое множество.

**bitset<bits>& bitset<bits>::operator <<=** (size\_t *num*)

- Сдвигает все биты влево на *num* позиций.
- Возвращает модифицированное битовое множество.
- Самый младший из *num* битов устанавливается равным false.

**bitset<bits>& bitset<bits>::operator >>=** (size\_t *num*)

- Сдвигает все биты вправо на *num* позиций.
- Возвращает модифицированное битовое множество.
- Самый старший из *num* битов устанавливается равным false.

## Доступ с помощью операции [ ]

**bitset<bits>::reference bitset<bits>::operator [ ]** (size\_t *idx*)

bool **bitset<bits>::operator [ ]** (size\_t *idx*) const

- Обе версии возвращают бит в позиции *idx*.
- Первая версия для неконстантных битов использует тип прокси, чтобы использовать возвращаемое значение как модифицируемое (lvalue). Подробнее см. ниже.
- Вызывающая сторона должна гарантировать, что параметр *idx* является действительным индексом; в противном случае поведение непредсказуемо.

Если операция [ ] применяется к неконстантным битовым множествам, то она возвращает специальный временный объект типа `bitset<>::reference`. Этот объект используется как прокси<sup>2</sup>, допускающий некоторые модификации с битами, доступ к которым

<sup>2</sup>Прокси позволяет сохранять контроль в тех местах, где он обычно не предусмотрен. Он часто используется для повышения безопасности. В данном случае прокси поддерживает контроль, чтобы позволить выполнение некоторых операций, хотя возвращаемое значение в принципе ведет себя как объект типа `bool`.

обеспечивается операцией [ ]. В частности, для объектов `reference` предусмотрены пять операций.

1. `reference& operator= (bool)`  
Устанавливает бит в соответствии с передаваемым значением.
2. `reference& operator= (const reference&)`  
Устанавливает бит в соответствии с другой ссылкой.
3. `reference& flip ()`  
Переключает значение бита.
4. `operator bool () const`  
Автоматически преобразовывает значение в булево.
5. `bool operator~ () const`  
Возвращает дополнение бита (его переключенное значение).

Например, можно написать следующие операторы:

```
bitset<50> flags;
...
flags[42] = true;           // установим бит 42
flags[13] = flags[42];      // присваиваем значение бита 42 биту 13
flags[42].flip();           // переключаем значение бита 42
if (flags[13]) {            // если бит 13 установлен,
    flags[10] = ~flags[42];  // то присваиваем дополнение бита 42 биту 10
}
```

## Создание новых модифицированных битовых множеств

**`bitset<bits> bitset<bits>::operator ~ () const`**

- Возвращает новое битовое множество, в котором все биты имеют значения, обратные значениям в объекте `*this`.

**`bitset<bits> bitset<bits>::operator << (size_t num) const`**

- Возвращает новое битовое множество, в котором все биты сдвинуты влево на `num` позиций.

**`bitset<bits> bitset<bits>::operator >> (size_t num) const`**

- Возвращает новое битовое множество, в котором все биты сдвинуты вправо на `num` позиций.

**`bitset<bits> operator & (const bitset<bits>& bits1,  
const bitset<bits>& bits2)`**

- Возвращает новое битовое множество, вычисленное с помощью операции И, примененной к объектам `bits1` и `bits2`.
- Возвращает новое битовое множество, в котором установлены только те биты, которые установлены как в объекте `bits1`, так и в объекте `bits2`.

**bitset<bits> operator |** (const bitset<bits>& *bits1*,  
const bitset<bits>& *bits2*)

- Возвращает новое битовое множество, вычисленное с помощью операции ИЛИ, примененной к объектам *bits1* и *bits2*.
- Возвращает новое битовое множество, в котором установлены только те биты, которые установлены или в объекте *bits1*, или в объекте *bits2* (или в обоих).

**bitset<bits> operator ^** (const bitset<bits>& *bits1*,  
const bitset<bits>& *bits2*)

- Возвращает новое битовое множество, вычисленное с помощью операции исключающего ИЛИ, примененной к объектам *bits1* и *bits2*.
- Возвращает новое битовое множество, содержащее только биты, установленные в объекте *bits1*, но не в объекте *bits2*, и наоборот.

## Операции преобразования типов

unsigned long **bitset<bits>::to\_ulong** () const

- Возвращает целочисленное значение, которое представляют биты битового множества.
- Если целочисленное значение невозможно представить типом unsigned long, генерирует исключение `overflow_error`.

unsigned long long **bitset<bits>::to\_ullong** () const

- Возвращает целочисленное значение, которое представляют биты битового множества.
- Если целочисленное значение невозможно представить типом unsigned long long, генерирует исключение `overflow_error`.
- Начиная со стандарта C++11.

string **bitset<bits>::to\_string** () const

string **bitset<bits>::to\_string** (charT *zero*) const

string **bitset<bits>::to\_string** (charT *zero*, charT *one*) const

- Возвращает строку, содержащую значение битового множества в двоичном виде, в котором символ '0' означает сброшенные биты и '1' — установленные.
- Порядок символов эквивалентен порядку битов с убывающими индексами.
- Параметры *zero* и *one* введены в стандарте C++11.
- Например:

```
bitset<50> b;
std::string s;
...
s = b.to_string();
```

- Это шаблонная функция, параметризованная только типом возвращаемого значения, у которой до принятия стандарта C++11 не было значения, заданного по умолчанию. Таким образом, по старым правилам языка C++11 следовало написать:

```
s = b.to_string<char, char_traits<char>, allocator<char> >();
```

## Операции ввода-вывода

*istream*& **operator** >> (*istream*& *strm*, *bitset*<*bits*>& *bits*)

- Считывает в объект *bits* битовое множество, представляющее собой последовательность символов '0' и '1'.
- Считывание выполняется до тех пор, пока не произойдет одно из следующих событий.
  - Считано *bits* символов.
  - В потоке *strm* обнаружен конец файла.
  - Следующий символ не совпадает ни с '0', ни с '1'.
- Возвращает *strm*.
- Если количество считанных битов меньше, чем количество битов в битовом множестве, то битовое множество заполняется ведущими нулями.
- Если невозможно считать ни один символ, то эта операция устанавливает в потоке *strm* флаг `ios::failbit`, который может активизировать соответствующее исключение (см. раздел 15.4.4).

*ostream*& **operator** << (*ostream*& *strm*, const *bitset*<*bits*>& *bits*)

- Записывает битовое множество *bits*, преобразованное в строку, содержащую двоичное представление (т.е. представляющую собой последовательность '0' и '1').
- Использует функцию `to_string()` (см. раздел S.1.2) для создания символов вывода.
- Возвращает поток *strm*.
- См. пример в разделе 12.5.1.

## Поддержка хеширования

После принятия стандарта C++11 класс `bitset<N>` имеет специализацию хеш-функции:

```
namespace std {
    template <size_t N> struct hash<bitset<N> >;
}
```

Подробнее см. в разделе 7.9.2.

## S.2. Массивы значений

Начиная со стандарта C++98, стандартная библиотека C++ содержит класс `valarray<T>` для обработки массивов, содержащих числовые значения. Массив значений — это представление математической концепции линейной последовательности значений.

Он имеет одну размерность, но создает иллюзию высокой размерности за счет специального способа вычисления индексов и мощные возможности выделения подмножеств. Следовательно, массив значений можно использовать и для векторных и матричных операций, и для эффективной обработки систем полиномиальных уравнений.

Классы массивов значений позволяют осуществлять сложную оптимизацию. Однако неясно, насколько важным окажется этот компонент стандартной библиотеки шаблонов

в будущем, потому что существуют более эффективные и интересные разработки. Одним из наиболее интересных примеров является система Blitz. Если вас интересуют вопросы числовой обработки, обратитесь к системе Blitz. Подробности изложены на веб-сайте по адресу <http://www.oonumerics.org/blitz/>.<sup>3</sup>

Классы массивов значений разработаны не очень хорошо. Фактически никто не может гарантировать, что их окончательная спецификация является работоспособной. Это произошло потому, что никто не чувствовал ответственности за эти классы. Люди, включившие массивы значений в стандартную библиотеку C++, вышли из комитета по стандартизации задолго до окончания работы над стандартом. Например, для того чтобы использовать массивы значений, часто приходится выполнять неудобные и длинные преобразования типов (см. раздел S.2.2).

## Новшества стандарта C++11

Стандарт C++98 определял практически все функциональные возможности массивов значений. Перечислим наиболее важные свойства, добавленные в стандарте C++11.

- Массивы значений поддерживают семантику перемещения. Для них существуют перемещающий конструктор и перемещающая операция присваивания.
- Класс содержит функции `swap()` (см. раздел S.2.3).
- Можно использовать список инициализации для заполнения массива значений начальными значениями или присваивания ему новых значений (см. раздел S.2.1).
- Класс массивов значений имеет функции `begin()` и `end()` для обхода своих элементов (см. раздел S.2.1).
- Операция `[]` возвращает константную ссылку, а не копию (см. раздел S.2.3).

### S.2.1. Описание массивов значений

Массивы значений — это одномерные массивы с элементами, последовательно пронумерованными начиная с нуля. Они реализуют возможность выполнять определенные числовые процедуры над всеми или частью значений в одном или нескольких массивах.

Например, можно выполнить оператор

$$z = a * x * x + b * x + c$$

в котором переменные `a`, `b`, `c`, `x` и `z` представляют собой массивы, содержащие сотни числовых значений, что позволяет упростить программу. Кроме того, эти операции над массивами значений выполняются эффективно, потому что классы выполняют определенную оптимизацию, чтобы избежать создания временных объектов при вычислении оператора. Кроме того, специальные интерфейсы и вспомогательные классы предоставляют возможность обрабатывать только одно определенное подмножество массивов значений или имитировать работу с многомерными массивами. Таким образом, массивы значений представляют собой концепцию, помогающую реализовать векторные и матричные операции, а также реализующие их классы.

<sup>3</sup> Актуальность ссылок не гарантируется. — *Примеч. ред.*



Стандарт гарантирует, что массивы значений не имеют псевдонимов. Иначе говоря, к любому значению неконстантного массива значений можно обратиться по уникальному пути. Таким образом, операции над этими значениями можно оптимизировать, потому что компилятор не учитывает данные, к которым обращаются по другому пути.

## Заголовочный файл

Массивы значений объявлены в заголовочном файле `<valarray>`.

```
#include <valarray>
```

В частности, в заголовочном файле `<valarray>` определены следующие классы:

```
namespace std {
    template<typename T> class valarray;           // массив значений типа T

    class slice;                                   // срез массива значений
    template<typename T> class slice_array;

    class gslice;                                 // обобщенный срез
    template<typename T> class gslice_array;

    template<typename T> class mask_array;        // массив значений с маской
    template<typename T> class indirect_array;    // перечисляемый массив значений
}
```

Классы имеют следующий смысл:

- `valarray` — основной класс, управляющий массивом значений;
- `slice` и `gslice` определяют срез как подмножество массива значений по аналогии с библиотекой BLAS;<sup>4</sup>
- `slice_array`, `gslice_array`, `mask_array` и `indirect_array` — это внутренние вспомогательные классы, используемые для хранения временных значений или данных. Эти классы невозможно использовать для непосредственного программирования интерфейса. Они создаются косвенно определенными операциями над массивами значений.

Все классы параметризованы шаблонным типом элементов. В принципе в качестве шаблонного параметра может использоваться любой тип данных. Однако по природе массивов значений это должен быть числовой тип.

## Создание массивов значений

При создании массива значений количество его элементов задается как параметр.

```
std::valarray<int> val(10);           // массив значений из десяти целых чисел
                                     // с нулевыми значениями
```

<sup>4</sup>Библиотека Basic Linear Algebra Subprograms (BLAS) содержит основные средства для реализации операций линейной алгебры, таких как умножение матриц, решение треугольных систем и простые векторные операции.

```
std::valarray<float> va2(5.7,10); // массив значений из десяти чисел типа
                                // float со значением 5.7

                                // (обратите внимание на порядок аргументов)
```

При передаче одного аргумента он интерпретируется как размер. Элементы инициализируются конструктором типа значений по умолчанию. Элементы массива значений, имеющие фундаментальный тип, инициализируются нулем (см. описание процесса инициализации данных фундаментального типа с помощью конструктора по умолчанию, приведенное в разделе 3.2.1). При передаче второго параметра первый параметр считается начальным значением элемента, а второй — количеством элементов. Отметим, что порядок передачи двух аргументов конструктору отличается от всех других классов в стандартной библиотеке C++. Все контейнерные классы STL используют первый числовой аргумент как количество членов, а второй — как начальное значение.

После принятия стандарта C++11 массив значений можно инициализировать с помощью списка инициализации.

```
std::valarray<int> va3 = { 3, 6, 18, 3, 22 };
```

До принятия стандарта C++11 необходимо было использовать обычный массив из языка C.

```
int array[] = { 3, 6, 18, 3, 22 };
// инициализируем массив значений элементами обычного массива
std::valarray<int> va3(array, sizeof(array)/sizeof(array[0]));

// инициализируем второй элемент из четырех
std::valarray<int> va4(array+1, 3);
```

Массив значений копирует передаваемое значение. Таким образом, для инициализации можно передавать временные данные.

## Операции над массивами значений

Для доступа к элементу массива значений определен оператор индексации массивов значений. Как обычно, первый индекс равен нулю.

```
va[0] = 3 * va[1] + va[2];
```

Кроме того, для массивов значений определены обычные числовые операции: сложение, вычитание, умножение, деление по модулю, отрицание, побитовые операции, операции сравнения и логические операции, а также все операции присваивания. Эти операции применяются к каждому элементу массива значений. Таким образом, результат операции над массивом значений — это массив значений, размер которого совпадает с размерами операндов, а элементы являются результатами поэлементного вычисления. Например, инструкция

```
val = va2 * va3;
```

эквивалентна

```
val[0] = va2[0] * va3[0];
val[1] = va2[1] * va3[1];
val[2] = va2[2] * va3[2];
...
```

Если количество элементов в комбинируемых массивах значений разное, результат становится неопределенным. Разумеется, можно выполнять только те операции, которые предусмотрены для элементов массивов значений. Точный смысл операции зависит от смысла операции над элементами. Таким образом, все эти операции просто делают одно и то же с каждым элементом или парой элементов обрабатываемого массива значений.

В бинарных операциях один из операндов может быть отдельным значением, тип которого совпадает с типом элементов массива. В этом случае отдельное значение комбинируется с каждым элементом массива значений, используемого в качестве второго операнда.

Например, инструкция

```
val = 4 * va2;
```

эквивалентна инструкциям

```
val[0] = 4 * va2[0];
val[1] = 4 * va2[1];
val[2] = 4 * va2[2];
...
```

Тип отдельного значения должен точно совпадать с типом элемента массива значений. Следовательно, предыдущий пример будет работоспособным только в том случае, если элементы имеют тип `int`. Следующая запись оказывается ошибочной:

```
std::valarray<double> va(20);
...
va = 4 * va; // ОШИБКА: несовпадение типов
```

Эта схема бинарных операций применима и к операциям сравнения. Таким образом, операция `==` *не возвращает* отдельное булево значение, означающее, что массивы значений равны. Вместо этого операция возвращает новый массив значений с тем же количеством элементов типа `bool`, в котором каждый элемент является результатом индивидуального сравнения. Например, в коде

```
std::valarray<double> va1(10);
std::valarray<double> va2(10);
std::valarray<bool> vab(10);
...
vab = (va1 == va2);
```

последняя инструкция эквивалентна записи

```
vab[0] = (va1[0] == va2[0]);
vab[1] = (va1[1] == va2[1]);
vab[2] = (va1[2] == va2[2]);
...
vab[9] = (va1[9] == va2[9]);
```

По этой причине невозможно упорядочить массив значений с помощью операции `<` и нельзя использовать такие массивы, как элементы контейнеров STL, если проверка равенства выполняется с помощью операции `==` (см. раздел 6.11.1, в котором описаны требования к элементам контейнеров STL).

Следующая программа демонстрирует простое использование массивов значений:

```
// num/valarray1.cpp

#include <iostream>
#include <valarray>
using namespace std;

// выводим массив значений
template <typename T>
void printValarray (const valarray<T>& va)
{
    for (int i=0; i<va.size(); i++) {
        cout << va[i] << ' ';
    }
    cout << endl;
}

int main()
{
    // определяем два массива значений с десятью элементами
    valarray<double> val(10), va2(10);

    // присваиваем значения 0.0, 1.1 до 9.9 первому массиву значений
    for (int i=0; i<10; i++) {
        val[i] = i * 1.1;
    }

    // присваиваем -1 всем элементам второго массива значений
    va2 = -1;

    // выводим оба массива значений
    printValarray(val);
    printValarray(va2);

    // выводим минимум, максимум и сумму элементов первого массива значений
    cout << "min(): " << val.min() << endl;
    cout << "max(): " << val.max() << endl;
    cout << "sum(): " << val.sum() << endl;

    // присваиваем значение первого массива значений второму
    va2 = val;

    // удаляем все элементы из первого массива значений
    val.resize(0);

    // снова выводим оба массива значений
    printValarray(val);
    printValarray(va2);
}
```

Программа выводит следующий результат:

```
0 1.1 2.2 3.3 4.4 5.5 6.6 7.7 8.8 9.9
-1 -1 -1 -1 -1 -1 -1 -1 -1 -1
min(): 0
```

```
max(): 9.9
sum(): 49.5
0 1.1 2.2 3.3 4.4 5.5 6.6 7.7 8.8 9.9
```

Начиная со стандарта C++11, для обработки элементов можно использовать интервальные циклы `for`, потому что массивы значений имеют глобальные функции `begin()` и `end()` (см. раздел S.2.3).

```
// выводим массив значений (начиная со стандарта C++11):
template <typename T>
void printValarray (const valarray<T>& va)
{
    for (const T& elem: va) {
        cout << elem << ' ';
    }
    cout << endl;
}
```

## Трансцендентные функции

Трансцендентные функции (тригонометрические и экспоненциальная) определены как эквиваленты числовых операций. Эти операции выполняются над всеми элементами массивов значений. В бинарных операциях один из операндов может быть отдельным значением, а вторым операндом — все значения массива значений.

Все эти операции определены как глобальные функции, а не как функции-члены, чтобы обеспечивать автоматическое преобразование типов для подмножеств массивов значений для обоих операндов (подмножества массивов значений описаны в разделе S.2.2).

Рассмотрим второй пример использования массивов значений. Он демонстрирует использование трансцендентных операций:

```
// num/valarray2.cpp

#include <iostream>
#include <valarray>
using namespace std;

// выводим массив значений
template <typename T>
void printValarray (const valarray<T>& va)
{
    for (int i=0; i<va.size(); i++) {
        cout << va[i] << ' ';
    }
    cout << endl;
}

int main()
{
    // создаем и инициализируем массив значений девятью элементами
    valarray<double> va(9);
    for (int i=0; i<va.size(); i++) {
```

```

        va[i] = i * 1.1;
    }

    // выводим массив значений
    printValarray(va);

    // удваиваем значения в массиве значений
    va *= 2.0;

    // снова выводим массив значений
    printValarray(va);

    // создаем второй массив значений, инициализированный значениями
    // первого массива, увеличенными на 10
    valarray<double> vb(va+10.0);

    // выводим второй массив значений
    printValarray(vb);

    // создаем третий массив значений как результат обработки
    // двух существующих массивов значений
    valarray<double> vc(9);
    vc = sqrt(va) + vb/2.0 - 1.0;

    // выводим третий массив значений
    printValarray(vc);
}

```

Программа выводит следующий результат:

```

0 1.1 2.2 3.3 4.4 5.5 6.6 7.7 8.8
0 2.2 4.4 6.6 8.8 11 13.2 15.4 17.6
10 12.2 14.4 16.6 18.8 21 23.2 25.4 27.6
4 6.58324 8.29762 9.86905 11.3665 12.8166 14.2332 15.6243 16.9952

```

## S.2.2. Подмножества массивов значений

Операция индексации [ ] перегружена для специальных вспомогательных объектов, создаваемых на основе массивов значений. Эти вспомогательные объекты определяют разнообразные подмножества массивов значений, обеспечивая элегантный способ работы с подмножествами массивов значений (с доступом для чтения и записи).

Подмножество массива значений определяется с помощью индекса. Рассмотрим пример:

```

va[std::slice(2,4,3)] // четыре элемента с расстоянием между ними, равным 3,
                        // начиная с индекса 2
va[va>7]              // все элементы со значением, большим 7

```

Если определение подмножества, например `std::slice(2,4,3)` или `va>7`, относится к константному массиву значений, выражение возвращает новый массив значений с соответствующими элементами. Однако, если определение подмножества относится к неконстантному массиву значений, то выражение возвращает временный объект специального

вспомогательного класса массивов значений. Этот временный объект содержит не значения подмножества, а его определение. Таким образом, вычисление значений откладывается до момента, когда потребуется окончательный результат.

Преимущество этого механизма, называемого *отложенным вычислением*, заключается в том, что он не вычисляет временные значения выражений. Это экономит время и память. Кроме того, этот метод поддерживает семантику ссылок. Таким образом, подмножества — это логические множества ссылок на исходные значения. Подмножество можно использовать как левостороннее значение оператора (lvalue). Например, подмножеству массива значений можно присвоить результат умножения двух других подмножеств одного и того же массива значений.

Однако, если элементы подмножества-получателя одновременно используются в подмножестве-источнике, могут возникнуть непредвиденные ситуации. Следовательно, любая операция над массивом значений работает корректно, только если элементы подмножества-получателя и элементы всех подмножеств-источников являются разными.

С помощью изощренных определений подмножеств можно придать массивам значений многомерную семантику. Таким образом, массивы значений можно использовать как многомерные массивы.

Существуют четыре способа определения массивов значений.

1. Срезы.
2. Обобщенные срезы.
3. Подмножества с масками.
4. Перечисляемые подмножества.

## Проблемы, связанные с подмножествами массивов значений

Прежде чем перейти к описанию подмножеств, рассмотрим одну общую проблему. Обработка подмножеств массивов значений плохо спроектирована. Создать подмножества легко, но комбинировать их с другими подмножествами трудно. К сожалению, почти всегда требуется явное преобразование типа в класс `valarray`. Причина заключается в том, что стандартная библиотека C++ не гарантирует, что подмножества массивов значений поддерживают те же операции, что и массивы значений.

Например, чтобы умножить два подмножества и присвоить результат третьему подмножеству, нельзя написать код

```
// ОШИБКА: нет преобразования
va[std::slice(0,4,3)] = va[std::slice(1,4,3)] * va[std::slice(2,4,3)];
```

Вместо этого необходимо написать код, использующий приведение

```
va[std::slice(0,4,3)]
= static_cast<std::valarray<double>>(va[std::slice(1,4,3)]) *
  static_cast<std::valarray<double>>(va[std::slice(2,4,3)]);
```

или приведение в старом стиле:

```
va[std::slice(0,4,3)]
= std::valarray<double>(va[std::slice(1,4,3)]) *
  std::valarray<double>(va[std::slice(2,4,3)]);
```

Это неудобно и повышает вероятность ошибок. Что еще хуже, без хорошей оптимизации производительность программы может снизиться, потому что каждое приведение создает временный объект, который можно было избежать без приведения.

Для того чтобы обработка была более удобной, можно применить следующую шаблонную функцию:

```
// шаблон для преобразования подмножества массива значений в массив значений
template <typename T>
inline
std::valarray<typename T::value_type> VA (const T& valarray_subset)
{
    return std::valarray<typename T::value_type>(valarray_subset);
}
```

Используя этот шаблон, можно было бы написать следующие инструкции:

```
va[std::slice(0,4,3)] = VA(va[std::slice(1,4,3)]) *
                      VA(va[std::slice(2,4,3)]); // OK
```

Однако проблемы с производительностью все равно остаются.

Если в программе используется определенный тип элементов, можно применить простое определение типа:

```
typedef valarray<double> VAD;
```

С помощью этого определения типа можно также написать операторы

```
va[std::slice(0,4,3)] = VAD(va[std::slice(1,4,3)]) *
                      VAD(va[std::slice(2,4,3)]); // OK
```

при условии, что элементы `va` имеют тип `double`.

## Срезы

Срез — это множество индексов, имеющий три следующих свойства.

1. Начальный индекс.
2. Количество элементов (размер).
3. Расстояние между элементами (шаг).

Эти три свойства можно передать точно в таком же порядке, как параметры конструктора класса `slice`. Например, следующее выражение определяет четыре элемента, начиная с индекса 2 и с шагом 3.

```
std::slice(2,4,3)
```

Иначе говоря, это выражение задает следующий набор индексов:

```
2 5 8 11
```

Шаг может быть отрицательным. Например, выражение

```
std::slice(9,5,-2)
```



задает индексы

9 7 5 3 1

Для того чтобы определить подмножество массива значений, можно использовать срез как аргумент операции индексации. Например, следующее выражение определяет подмножество массива значений `va`, содержащее элементы с индексами 2, 5, 8 и 11.

```
va[std::slice(2,4,3)]
```

Вызывающая сторона должна гарантировать, что все эти индексы являются корректными.

Если подмножество квалифицировано как срез константного массива значений, то оно считается новым массивом значений. Если массив значений не является константным, то подмножество содержит ссылки на исходный массив значений. Вспомогательный класс `slice_array` используется следующим образом:

```
namespace std {
    class slice;

    template <typename T>
    class slice_array;

    template <typename T>
    class valarray {
    public:
        // срез константного массива значений возвращает новый массив значений
        valarray<T> operator[] (slice) const;

        // срез переменного числового массива возвращает объект класса slice_array
        slice_array<T> operator[] (slice);
        ...
    };
}
```

Для объектов класса `slice_array` определены следующие операции.

- Присваивание отдельного значения всем элементам.
- Присваивание другого массива значений (или подмножества массива значений).
- Вызов любой вычисляемой операции присваивания, например `+=` или `*=`.

Для любой другой операции требуется преобразование подмножества в массив значений (см. раздел S.2.2). Класс `slice_array<>` является внутренним вспомогательным классом для срезов и должен быть доступным для пользователя. Таким образом, все конструкторы и операции присваивания в классе `slice_array<>` являются закрытыми.

Например, оператор

```
va[std::slice(2,4,3)] = 2;
```

присваивает 2 третьему, шестому, девятому и двенадцатому элементам массива значений `va`. Это эквивалентно следующим операторам:

```
va[2] = 2;
va[5] = 2;
va[8] = 2;
va[11] = 2;
```

В качестве другого примера рассмотрим следующий оператор, который возводит в квадрат значения элементов с индексами 2, 5, 8 и 11.

```
va[std::slice(2,4,3)]
```

```
*= std::valarray<double>(va[std::slice(2,4,3)]);
```

Как указано в разделе S.2.2, нельзя просто написать

```
va[std::slice(2,4,3)] *= va[std::slice(2,4,3)]; // ОШИБКА
```

Однако, используя шаблонную функцию `VA()`, упомянутую в разделе S.2.2, можно написать

```
va[std::slice(2,4,3)] *= VA(va[std::slice(2,4,3)]); // OK
```

Передавая разные срезы одного и того же массива значений, можно комбинировать разные подмножества и сохранять результаты в других подмножествах массива значений. Например, выражение

```
va[std::slice(0,4,3)] = VA(va[std::slice(1,4,3)]) *  
                        VA(va[std::slice(2,4,3)]);
```

эквивалентно следующим инструкциям:

```
va[0] = va[1] * va[2];  
va[3] = va[4] * va[5];  
va[6] = va[7] * va[8];  
va[9] = va[10] * va[11];
```

Если рассматривать массив значений как двумерную матрицу, то этот пример представляет собой умножение векторов (рис. S.1). Однако порядок отдельных операций присваивания не определен. Следовательно, поведение становится неопределенным, если подмножество-получатель содержит элементы, используемые в подмножествах-источниках.

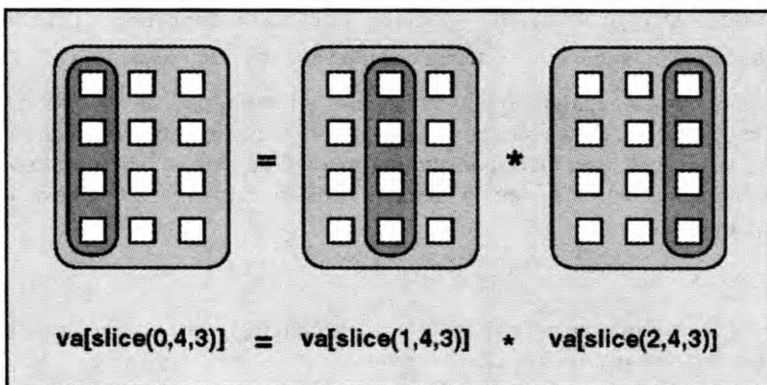


Рис. S.1. Умножение срезов массива значений

Таким способом можно создавать еще более сложные выражения:

```
va[std::slice(0,100,3)]
    = std::pow(VA(va[std::slice(1,100,3)]) * 5.0,
               VA(va[std::slice(2,100,3)]));
```

Отметим еще раз, что отдельное значение, в данном случае 5.0, должно точно соответствовать типу элементов массива значений

Следующая программа демонстрирует законченный пример использования срезов массивов значений:

```
// num/slicel.cpp

#include <iostream>
#include <valarray>
using namespace std;

// построчный вывод массива значений
template <typename T>
void printValarray (const valarray<T>& va, int num)
{
    for (int i=0; i<va.size()/num; ++i) {
        for (int j=0; j<num; ++j) {
            cout << va[i*num+j] << ' ';
        }
        cout << endl;
    }
    cout << endl;
}

int main()
{
    // массив значений из 12 элементов
    // - четыре строки
    // - три столбца
    valarray<double> va(12);

    // заполняем массив значений значениями
    for (int i=0; i<12; i++) {
        va[i] = i;
    }

    printValarray (va, 3);

    // первая строка = второй столбец, возведенный в степень,
    // стоящую в третьем столбце
    va[slice(0,4,3)] = pow (valarray<double>(va[slice(1,4,3)]),
                           valarray<double>(va[slice(2,4,3)]));
    printValarray (va, 3);

    // создаем числовой массив, в котором трижды повторяется третий элемент
    // массива значений va
    valarray<double> vb(va[slice(2,4,0)]);
```

```
// умножаем третий столбец на элементы массива значений vb
va[slice(2,4,3)] *= vb;

printValarray (va, 3);

// выводим квадратный корень из элементов второй строки
printValarray (sqrt(valarray<double>(va[slice(3,3,1)])));

// удваиваем элементы третьей строки
va[slice(2,4,3)] = valarray<double>(va[slice(2,4,3)]) * 2.0;

printValarray (va, 3);
}
```

Программа выводит следующие результаты:

```
0 1 2
3 4 5
6 7 8
9 10 11

1 1 2
1024 4 5
5.7648e+006 7 8
1e+011 10 11

1 1 4
1024 4 10
5.7648e+006 7 16
1e+011 10 22
32 2 3.16228

1 1 8
1024 4 20
5.7648e+006 7 32
1e+011 10 44
```

## Обобщенные срезы

*Обобщенные срезы* (general slices, или *gslices*) — это универсальная форма срезов. Подобно обычным срезам, позволяющим обрабатывать одномерное подмножество, обобщенные срезы позволяют работать с подмножествами многомерных массивов. В принципе обобщенные срезы имеют те же свойства, что и обычные срезы.

- Начальный индекс.
- Количество элементов (размер).
- Расстояние между элементами (шаг).

Тем не менее, в отличие от обычных срезов, в обобщенных срезах количество элементов и шаг являются массивами значений. Количество элементов в таком массиве равно количеству используемых размерностей. Например, обобщенный срез, имеющий состояние

```
start: 2
size: [ 4 ]
stride: [ 3 ]5
```

эквивалентен обычному срезу, потому что этот массив имеет одну размерность. Таким образом, обобщенные срезы определяют четыре элемента с шагом 3, начиная с индекса 2:

```
2 5 8 11
```

Однако обобщенный срез, имеющий состояние

```
start: 2
size: [ 2 4 ]
stride: [ 10 3 ]
```

имеет две размерности. Наименьший индекс имеет высшую размерность. Таким образом, этот обобщенный срез задает начало с индексом 2, два элемента с шагом 10 и четыре элемента с шагом 3:

```
2 5 8 11
12 15 18 21
```

Рассмотрим пример среза с тремя размерностями:

```
start: 2
size: [ 3 2 4 ]
stride: [ 30 10 3 ]
```

Он задает начало с индексом 2, три элемента с шагом 30, два элемента с шагом 10 и четыре элемента с шагом 3:

```
2 5 8 11
12 15 18 21

32 35 38 41
42 45 48 51

62 65 68 71
72 75 78 81
```

Единственное отличие обобщенных срезов от обычных — это возможность использовать массивы для определения размеров и шагов. Помимо этого, обобщенные срезы ничем не отличаются от обычных срезов.

1. Для определения конкретного подмножества массива значений достаточно передать обобщенный срез операции индексации массива значений.
2. Если массив значений является константным, то результатом выражения является новый массив значений.
3. Если массив значений является неконстантным, то результирующим выражением является объект класса `gslice_array`, представляющий элементы массива значений с семантикой ссылок:

<sup>5</sup>Stride — расстояние между элементами. — *Примеч. ред.*

```
namespace std {
    class gslice;

    template <typename T>
    class gslice_array;

    template <typename T>
    class valarray {
    public:
        // обобщенный срез константного массива значений
        // возвращает новый массив значений
        valarray<T> operator[] (const gslice&) const;

        // обобщенный срез переменного массива значений
        // возвращает объект класса gslice_array
        gslice_array<T> operator[] (const gslice&);
        ...
    };
}
```

4. Для объектов класса `gslice_array` предусмотрены присваивание и вычисляемые операции присваивания, модифицирующие элементы подмножества.
5. С помощью преобразования типов можно комбинировать обобщенный срез с другими массивами значений и подмножествами массивов значений (см. раздел S.2.2).

Использование обобщенных срезов массивов значений демонстрирует следующая программа:

```
// num/gslice1.cpp

#include <iostream>
#include <valarray>
using namespace std;

// построчный вывод трехмерного массива значений
template <typename T>
void printValarray3D (const valarray<T>& va, int dim1, int dim2)
{
    for (int i=0; i<va.size()/(dim1*dim2); ++i) {
        for (int j=0; j<dim2; ++j) {
            for (int k=0; k<dim1; ++k) {
                cout << va[i*dim1*dim2+j*dim1+k] << ' ';
            }
            cout << '\n';
        }
        cout << '\n';
    }
    cout << endl;
}

int main()
{
    // массив значений из 24 элементов
    // - две группы
```

```

// - четыре строки
// - три столбца
valarray<double> va(24);

// заполняем массив значениями
for (int i=0; i<24; i++) {
    va[i] = i;
}

// выводим массив значений
printValarray3D (va, 3, 4);

// нам необходимы два двумерных подмножества, в которых
// записаны три тройки в двух 12-элементах массивах
size_t lengthvalues[] = { 2, 3 };
size_t stridevalues[] = { 12, 3 };
valarray<size_t> length(lengthvalues,2);
valarray<size_t> stride(stridevalues,2);

// присваиваем второй столбец первых трех строк
// первому столбцу первых трех строк
va[gslice(0,length,stride)]
    = valarray<double>(va[gslice(1,length,stride)]);

// добавляем и присваиваем третью из первых трех строк
// первой из первых трех строк
va[gslice(0,length,stride)]
    += valarray<double>(va[gslice(2,length,stride)]);

// выводим массив значений
printValarray3D (va, 3, 4);
}

```

Программа выводит следующие результаты:

```

0 1 2
3 4 5
6 7 8
9 10 11

12 13 14
15 16 17
18 19 20
21 22 23

3 1 2
9 4 5
15 7 8
9 10 11

27 13 14
33 16 17
39 19 20
21 22 23

```

## Маскированные подмножества

Еще один способ определить подмножество массива значений обеспечивают маскированные массивы. Накладывать маску на элементы можно с помощью булевых выражений. Например, в выражении

```
va[va > 7]
```

подвыражение

```
va > 7
```

определяет массив значений размера `va`, в котором булево значение указывает, имеет ли данный элемент значение, большее 7. Операция индексации использует булев массив значений для определения всех элементов, для которых булево выражение возвращает значение `true`. Таким образом, в массиве значений `va` выражение

```
va[va > 7]
```

задает подмножество элементов, которые больше 7.

Помимо этого, маскированные массивы ведут себя точно так же, как и все остальные подмножества массива значений.

1. Для определения конкретного подмножества массива значений достаточно передать массив значений булевых значений операции индексации массива значений.
2. Если массив значений является константным, то результатом выражения становится новый массив значений.
3. Если массив значений является неконстантным, то результатом выражения является объект класса `mask_array`, представляющий элементы массива значений с семантикой ссылок:

```
namespace std {
    template <typename T>
    class mask_array;

    template <typename T>
    class valarray {
    public:
        // наложение маски на константный массив значений
        valarray<T> operator[] (const valarray<bool>&) const;

        // masking a variable valarray returns a mask_array
        mask_array<T> operator[] (const valarray<bool>&);
        ...
    };
}
```

4. Для объектов класса `mask_array` предусмотрены присваивание и вычисляемые операции присваивания, модифицирующие элементы подмножества.
5. С помощью преобразования типов можно комбинировать маскированные массивы и другие массивы значений, а также подмножества массивов значений (см. раздел S.2.2).



Использование маскированных массивов демонстрирует следующая программа:

```
// num/maskarray1.cpp

#include <iostream>
#include <valarray>
using namespace std;

// построчный вывод массива значений
template <typename T>
void printValarray (const valarray<T>& va, int num)
{
    for (int i=0; i<va.size()/num; ++i) {
        for (int j=0; j<num; ++j) {
            cout << va[i*num+j] << ' ';
        }
        cout << endl;
    }
    cout << endl;
}

int main()
{
    // массив значений из 12 элементов
    // - четыре строки
    // - три столбца
    valarray<double> va(12);

    // заполняем массив значений
    for (int i=0; i<12; i++) {
        va[i] = i;
    }

    printValarray (va, 3);

    // присваиваем 77 всем значениям, которые меньше 5
    va[va<5.0] = 77.0;

    // добавляем 100 ко всем значениям, которые больше 5 и меньше 9
    va[va>5.0 && va<9.0]
        = valarray<double>(va[va>5.0 && va<9.0]) + 100.0;

    printValarray (va, 3);
}
```

Программа выводит следующий результат:

```
0 1 2
3 4 5
6 7 8
9 10 11

77 77 77
```

```
77 77 5
106 107 108
9 10 11
```

Типы сравниваемых числовых значений из маскированных массивов должны точно совпадать. Например, значение `int`, которое используется для сравнения со значениями типа `double` из массива значений, является неприемлемым:

```
valarray<double> va(12);
...
va[va<5] = 77; // ОШИБКА
```

## Перечисляемые подмножества

Перечисляемые массивы обеспечивают четвертый, и последний, способ определения подмножества массива значений. В данном случае можно просто определить подмножество массива значений, передав массив индексов. Индексы, задающие подмножество, могут быть не упорядоченными и указываться дважды.

Во всем остальном перечисляемые массивы ничем не отличаются от обычных подмножеств.

1. Для определения конкретного подмножества массива значений достаточно передать массив элементов типа `size_t` операции индексации массива значений.
2. Если массив значений является константным, то результатом выражения является новый массив значений.
3. Если массив значений является неконстантным, то результатом выражения является объект класса `indirect_array`, представляющий элементы массива значений с семантикой ссылок.

```
namespace std {
    template <typename T>
    class indirect_array;

    template <typename T>
    class valarray {
    public:
        // операция индексации константного массива значений возвращает
        // новый массив значений
        valarray<T> operator[] (const valarray<size_t>&) const;

        // операция индексации переменного массива значений возвращает
        // объект класса indirect_array
        indirect_array<T> operator[] (const valarray<size_t>&);
        ...
    };
}
```

4. Для объектов класса `indirect_array` предусмотрены присваивание и вычисляемые операции присваивания, модифицирующие элементы подмножества.
5. С помощью преобразования типов можно комбинировать перечислимые массивы и другие массивы значений, а также подмножества массивов значений (см. раздел S.2.2).

Использование перечислимых массивов демонстрирует следующая программа:

```
// num/indirectarray1.cpp

#include <iostream>
#include <valarray>
using namespace std;

// выводим массив значений как двумерный массив
template <typename T>
void printValarray (const valarray<T>& va, int num)
{
    for (int i=0; i<va.size()/num; i++) {
        for (int j=0; j<num; j++) {
            cout << va[i*num+j] << ' ';
        }
        cout << endl;
    }
    cout << endl;
}

int main()
{
    // создаем массив значений из 12 элементов
    valarray<double> va(12);

    // инициализируем массив значений числами 1.01, 2.02, ... 12.12
    for (int i=0; i<12; i++) {
        va[i] = (i+1) * 1.01;
    }

    printValarray(va,4);

    // создаем массив индексов
    // - примечание: элементы должны иметь тип size_t
    valarray<size_t> idx(4);
    idx[0] = 8;
    idx[1] = 0;
    idx[2] = 3;
    idx[3] = 7;

    // используем массив значений для вывода девятого, первого,
    // четвертого и восьмого элементов
    printValarray(valarray<double>(va[idx]), 4);

    // изменяем первый и четвертый элементы и выводим их косвенным образом
    va[0] = 11.11;
    va[3] = 44.44;
    printValarray(valarray<double>(va[idx]), 4);

    // теперь выбираем второй, третий, шестой и девятый элементы
    // и присваиваем им 99
    idx[0] = 1;
```

```
idx[1] = 2;
idx[2] = 5;
idx[3] = 8;
va[idx] = 99;

// снова выводим весь массив значений
printValarray (va, 4);
}
```

Массив значений `idx` используется для определения подмножества элементов в массиве значений `va`. Программа выводит следующие результаты:

```
1.01 2.02 3.03 4.04
5.05 6.06 7.07 8.08
9.09 10.1 11.11 12.12

9.09 1.01 4.04 8.08

9.09 11.11 44.44 8.08

11.11 99 99 44.44
5.05 99 7.07 8.08
99 10.1 11.11 12.12
```

### S.2.3. Подробное описание класса `valarray`

В основе массивов значений лежит шаблонный класс `valarray<>`. Он параметризован типом элементов:

```
namespace std {
    template <typename T>
    class valarray;
}
```

Размер не является частью типа. Следовательно, в принципе можно обрабатывать массивы значений разных размеров, а также изменять эти размеры. Однако изменение размера массива значений предназначено только для реализации двухэтапной инициализации (создание и установка размера), необходимой для управления массивами массивов значений. Результат комбинации массивов значений с разными размерами не определен.

### Создание, копирование и уничтожение

**`valarray::valarray ()`**

- Конструктор по умолчанию.
- Создает пустой массив значений.
- Этот конструктор предназначен только для создания массивов значений. На следующем этапе их правильный размер задается функцией-членом `resize()`.

**`explicit valarray::valarray (size_t num)`**

- Создает массив значений, содержащий *num* элементов.

- Элементы инициализируются их конструкторами по умолчанию. Элементы фундаментальных типов инициализируются нулями 0.

**valarray**: **valarray** (*initializer-list*)

- Создает массив значений, содержащий значения из списка *инициализации*.
- Начиная со стандарта C++11

**valarray**: **valarray** (const T& *value*, size\_t *num*)

- Создает массив значений, содержащий *num* элементов.
- Элемент инициализируется значением *value*.
- Порядок параметров необычен. Все другие классы в стандартной библиотеке C++ имеют интерфейс, в котором значение *num* является первым параметром, а значение *value* — вторым.

**valarray**: **valarray** (const T\* *array*, size\_t *num*)

- Создает массив значений, содержащий *num* элементов.
- Элементы инициализируются значениями элементом массива *array*.
- Вызывающая сторона должна гарантировать, что массив *array* содержит *num* элементов; в противном случае поведение не определено.

**valarray**: **valarray** (const *valarray*& *va*)

**valarray**: **valarray** (*valarray*&& *va*)

- Копирующие и перемещающий конструкторы.
- Создает массив значений как копию массива *va* или как массив, содержащий перемещенные элементы массива *va*.
- Перемещающий конструктор предусмотрен стандартом C++11.

**valarray**: **~valarray** ()

- Деструктор.
- Уничтожает все элементы и освобождает память.

Можно также создать массивы значений, инициализированные объектами внутренних вспомогательных классов `slice_array`, `gslice_array`, `mask_array` и `indirect_array`.

## Операции присваивания

*valarray*& **valarray**: **operator** = (const *valarray*& *va*)

*valarray*& **valarray**: **operator** = (*valarray*&& *va*)

- Операция копирующего или перемещающего присваивания элементов массива значений *va*.
- Если массив *va* имеет другой размер, поведение не определено.
- Значение элемента в левой части операции присваивания массивов значений не должно зависеть от значения другого элемента в левой части. Иначе говоря, если присваивание перезаписывает значения, используемые в правой части присваивания,

то результат будет не определен. Это значит, что не следует использовать элемент из левой части в выражении правой части. Это объясняется тем, что порядок вычисления операторов над массивами значений не определен (см. раздел S.2.2).

- Перемещающая операция присваивания предусмотрена стандартом C++11.

```
void valarray::swap (valarray& va2)
void swap (valarray& va1, valarray& va2)
```

- Обе версии обменивают значения двух массивов значений.
  - Функция-член обменивает содержимое массивов *\*this* и *va2*.
  - Глобальная функция обменивает содержимое массивов *va1* и *va2*.
- По возможности следует выбирать эти функции, а не операции присваивания, потому что они выполняются быстрее. Фактически они имеют константную сложность.
- Предусмотрены стандартом C++11.

```
valarray& valarray::operator = (initializer-list)
```

- Присваивает значения из *списка инициализации*.
- Возвращает *\*this*.
- Начиная со стандарта C++11.

```
valarray& valarray::operator = (const T& value)
```

- Присваивает значение *value* каждому элементу массива значений.
- Размер массива значений не изменяется. Указатели и ссылки на элементы остаются корректными.

Кроме того, можно присваивать объекты внутренних вспомогательных классов `slice_array`, `gslice_array`, `mask_array` и `indirect_array`.

## Функции-члены

В класс `valarray` входят следующие функции-члены:

```
size_t valarray::size () const
```

- Возвращает текущее количество элементов.

```
void valarray::resize (size_t num)
void valarray::resize (size_t num, T value)
```

- Обе версии изменяют размер массива значений *num*.
- Если размер увеличивается, то новые элементы инициализируются их конструкторами по умолчанию или значением *value* соответственно.
- Обе версии делают некорректными все указатели и ссылки на элементы массива значений.
- Эти функции предназначены только для создания массивов значений. После их создания с помощью конструктора по умолчанию необходимо установить их правильные размеры, вызвав указанные функции.

T **valarray::min** () const

T **valarray::max** () const

- Первая версия возвращает минимальное значение всех элементов.
- Вторая версия возвращает максимальное значение всех элементов.
- Элементы сравниваются с помощью операции < или > соответственно. Таким образом, эти операции должны быть предусмотрены типом элементов.
- Если массив значений не содержит элементов, то возвращаемое значение не определено.

T **valarray::sum** () const

- Возвращает сумму всех элементов.
- Элементы обрабатываются с помощью операции +=. Таким образом, эта операция должна быть предусмотрена типом элементов.
- Если массив значений не содержит элементов, то возвращаемое значение не определено.

*valarray* **valarray::shift** (int *num*) const

- Возвращает новый массив значений, в котором все элементы сдвинуты на *num* позиций.
- Возвращаемый массив значений имеет такое же количество элементов.
- Элементы на сдвинутых позициях инициализируются их конструкторами по умолчанию.
- Направление сдвига зависит от знака параметра *num*.
  - Если *num* — положительное число, то сдвиг выполняется влево, к началу. Таким образом, элементы получают меньшие индексы.
  - Если *num* — отрицательное число, то сдвиг выполняется вправо, к концу. Таким образом, элементы получают большие индексы.

*valarray* **valarray::cshift** (int *num*) const

- Возвращает новый массив значений, в котором все элементы циклически сдвинуты на *num* позиций.
- Возвращаемый массив значений имеет такое же количество элементов.
- Направление сдвига зависит от знака параметра *num*.
  - Если *num* — положительное число, то сдвиг выполняется влево, к началу. Таким образом, элементы получают меньшие индексы или вставляются в конец массива.
  - Если *num* — отрицательное число, то сдвиг выполняется вправо, к концу. Таким образом, элементы получают большие индексы или вставляются в начало.

*valarray* **valarray::apply** (T op(T)) const

*valarray* **valarray::apply** (T op(const T&)) const

- Обе версии возвращают новый массив значений, в котором ко всем элементам применена операция *op* ().

- Возвращаемый массив значений имеет такое же количество элементов.
- Для каждого элемента массива `*this` вызывается операция `op(elem)`, инициализирующая соответствующий элемент в новом возвращаемом массиве значений ее результатом.

## Доступ к элементам

```
T& valarray::operator[ ] (size_t idx)
const T& valarray::operator[ ] (size_t idx) const
```

- Обе версии возвращают элемент массива значений с индексом `idx` (первый элемент имеет индекс 0).
- Неконстантная версия возвращает ссылку. Таким образом, можно модифицировать элемент, заданный и возвращенный этой операцией. Гарантируется, что ссылка является корректной, пока массив значений существует и не вызываются функции, изменяющие его размер.
- До принятия стандарта C++11 значение, возвращаемое второй версией, имело тип `T`.

```
iterator begin (valarray& va)
const_iterator begin (const valarray& va)
```

- Обе версии возвращают итератор произвольного доступа, установленный на начало массива значений (на позицию первого элемента).
- Имя типа итератора не определено.
- Если контейнер пустой, то вызовы этих функций эквивалентны вызову `valarray::end()`.
- Начиная со стандарта C++11

```
iterator end (valarray& va)
const_iterator end (const valarray& va)
```

- Обе версии возвращают итератор произвольного доступа, установленный на конец контейнера (на позицию последнего элемента).
- Имя типа итератора не определено.
- Если контейнер пустой, то вызовы этих функций эквивалентны вызову `valarray::begin()`.
- Начиная со стандарта C++11.

## Операции над массивами значений

Унарные операции над массивами значений имеют следующий формат:

```
valarray valarray::unary-op () const
```

- Унарная операция возвращает новый массив значений, содержащий все значения массива `*this`, модифицированные операцией `unary-op`.
- Операцией `unary-op` может быть одна из следующих операций:



```
operator +
operator -
operator ~
operator !
```

- Значение, возвращаемое операцией `!`, имеет тип `valarray<bool>`.

Бинарные операции над массивами значений (за исключением операций сравнения и присваивания) имеют следующий формат:

```
valarray binary-op (const valarray& val, const valarray& va2)
valarray binary-op (const valarray& va, const T& value)
valarray binary-op (const T& value, const valarray& va)
```

- Эти операции возвращают новый массив значений с тем же количеством элементов, что и `va`, `val` или `va2`. Новый массив значений содержит результат применения операции **binary-op** к каждой паре значений.
- Если передается только один операнд в виде единственного значения `value`, то этот операнд комбинируется с каждым элементом массива `va`.
- Операцией **binary-op** может быть одна из следующих операций:

```
operator +
operator -
operator *
operator /
operator %
operator ^
operator &
operator |
operator <<
operator >>
```

- Если массивы `val` и `va2` имеют разное количество элементов, то результат не определен.

Логические операции и операции сравнения имеют аналогичную структуру. Однако они возвращают значения булева типа.

```
valarray<bool> logical-op (const valarray& val, const valarray& va2)
valarray<bool> logical-op (const valarray& va, const T& value)
valarray<bool> logical-op (const T& value, const valarray& va)
```

- Эти операции возвращают новый массив значений с тем же количеством элементов, что и `va`, `val` или `va2`. Новый массив значений содержит результат применения операции **logical-op** к каждой паре значений.
- Если передается только один операнд в виде единственного значения `value`, то он комбинируется с каждым элементом массива `va`.
- Операцией **logical-op** может быть одна из следующих операций:

```
operator ==
operator !=
operator <
operator <=
operator >
```

```
operator >=
operator &&
operator ||
```

- Если массивы *va1* и *va2* имеют разное количество элементов, то результат не определен.

Для массивов значений предусмотрены вычисляемые операции присваивания.

```
valarray& valarray::assign-op (const valarray& va)
valarray& valarray::assign-op (const T& value)
```

- Обе версии применяют операцию ***assign-op*** к каждому элементу массива *\*this* и соответствующему элементу массива *va* или *value* соответственно, переданному как второй операнд.
- Функции возвращают ссылку на модифицированный массив значений.
- Операцией ***assign-op*** может быть одна из следующих операций:

```
operator +=
operator -=
operator *=
operator /=
operator %=
operator &=
operator |=
operator ^=
operator <<=
operator >>=
```

- Если массивы *\*this* и *va2* содержат разное количество элементов, то результат не определен.
- Ссылки и указатели на модифицированные элементы остаются корректными, пока массив значений существует и не вызывается функции, изменяющие его размер.

## Трансцендентные функции

```
valarray abs (const valarray& va)
valarray pow (const valarray& va1, const valarray& va2)
valarray pow (const valarray& va, const T& value)
valarray pow (const T& value, const valarray& va)
valarray exp (const valarray& va)
valarray sqrt (const valarray& va)
valarray log (const valarray& va)
valarray log10 (const valarray& va)
valarray sin (const valarray& va)
valarray cos (const valarray& va)
valarray tan (const valarray& va)
valarray sinh (const valarray& va)
valarray cosh (const valarray& va)
valarray tanh (const valarray& va)
valarray asin (const valarray& va)
```

```

valarray acos (const valarray& va)
valarray atan (const valarray& va)
valarray atan2 (const valarray& va1, const valarray& va2)
valarray atan2 (const valarray& va, const T& value)
valarray atan2 (const T& value, const valarray& va)

```

- Все эти функции возвращают новый массив значений с тем же количеством элементов, что и *va*, *va1* и *va2*. Новый массив значений содержит результат соответствующей операции, примененной к каждому элементу или паре элементов.
- Если массивы *va1* и *va2* имеют разное количество элементов, то результат не определен.

## S.2.4. Подробное описание классов подмножеств массивов значений

Этот подраздел посвящен подробному описанию классов подмножеств массивов значений. Эти классы очень просты и не содержат много операций. По этой причине в подразделе приводятся только объявления и несколько замечаний.

### Классы `slice` и `slice_array`

Объекты класса `slice_array` создаются с помощью класса `slice`, используемого как индекс неконстантного массива значений.

```

namespace std {
    template <typename T>
    class valarray {
    public:
        ...
        slice_array<T> operator[] (slice);
        ...
    };
}

```

Точное определение открытого интерфейса класса `slice` имеет следующий вид:

```

namespace std {
    class slice {
    public:
        slice (); // пустое подмножество
        slice (size_t start, size_t size, size_t stride);
        size_t start() const;
        size_t size() const;
        size_t stride() const;
    };
}

```

Конструктор по умолчанию создает пустое подмножество. С помощью функций-членов `start()`, `size()` и `stride()` можно определить свойства среза.

Класс `slice_array` имеет следующие операции:

```
namespace std {
    template <typename T>
    class slice_array {
    public:
        typedef T value_type;

        void operator= (const T&);
        void operator= (const valarray<T>&) const;
        void operator*= (const valarray<T>&) const;
        void operator/= (const valarray<T>&) const;
        void operator%= (const valarray<T>&) const;
        void operator+= (const valarray<T>&) const;
        void operator-= (const valarray<T>&) const;
        void operator^= (const valarray<T>&) const;
        void operator&= (const valarray<T>&) const;
        void operator|= (const valarray<T>&) const;
        void operator<=<= (const valarray<T>&) const;
        void operator>=>= (const valarray<T>&) const;
        ~slice_array();
    private:
        slice_array();

        slice_array(const slice_array&);
        slice_array& operator=(const slice_array&);
        ...
    };
}
```

Класс `slice_array<>` предназначен исключительно для внутренних вспомогательных целей и должен быть доступен пользователю. Таким образом, все конструкторы и операция присваивания класса `slice_array<>` являются закрытыми.

## Классы `gslice` и `gslice_array`

Объекты класса `gslice_array` создаются с помощью класса `gslice`, используемого в качестве индекса неконстантного массива значений.

```
namespace std {
    template <typename T>
    class valarray {
    public:
        ...
        gslice_array<T> operator[] (const gslice&);
        ...
    };
}
```

Точное определение открытого интерфейса класса `gslice` имеет следующий вид:

```
namespace std {
    class gslice {
    public:
        gslice ();                // пустое подмножество
```

```

        gslice (size_t start,
                const valarray<size_t>& size,
                const valarray<size_t>& stride);
        size_t start() const;
        valarray<size_t> size() const;
        valarray<size_t> stride() const;
    };
}

```

Конструктор по умолчанию создает пустое подмножество. С помощью функций-членов `start()`, `size()` и `stride()` можно определить свойства обобщенного среза.

Класс `gslice_array` содержит следующие операции:

```

namespace std {
    template <typename T>
    class gslice_array {
    public:
        typedef T value_type;

        void operator= (const T&);
        void operator= (const valarray<T>&) const;
        void operator*= (const valarray<T>&) const;
        void operator/= (const valarray<T>&) const;
        void operator%= (const valarray<T>&) const;
        void operator+= (const valarray<T>&) const;
        void operator-= (const valarray<T>&) const;
        void operator^= (const valarray<T>&) const;
        void operator&= (const valarray<T>&) const;
        void operator|= (const valarray<T>&) const;
        void operator<=<= (const valarray<T>&) const;
        void operator>>= (const valarray<T>&) const;
        ~gslice_array();
    private:
        gslice_array();
        gslice_array(const gslice_array<T>&);
        gslice_array& operator=(const gslice_array<T>&);
        ...
    };
}

```

Как и класс `slice_array<>`, класс `gslice_array<>` предназначен исключительно для вспомогательных внутренних целей и должен быть доступен пользователю. Таким образом, все конструкторы и операция присваивания в классе `gslice_array<>` являются закрытыми.

## Класс `mask_array`

Объекты класса `mask_array` создаются с помощью класса `valarray<bool>`, используемого в качестве индекса неконстантного массива значений.

```

namespace std {
    template <typename T>
    class valarray {
    public:

```

```

        ...
        mask_array<T> operator[] (const valarray<bool>&);
        ...
    };
}

```

Класс `mask_array` имеет следующие операции:

```

namespace std {
    template <typename T>
    class mask_array {
    public:
        typedef T value_type;

        void operator= (const T&);
        void operator= (const valarray<T>&) const;
        void operator*= (const valarray<T>&) const;
        void operator/= (const valarray<T>&) const;
        void operator%= (const valarray<T>&) const;
        void operator+= (const valarray<T>&) const;
        void operator-= (const valarray<T>&) const;
        void operator^= (const valarray<T>&) const;
        void operator&= (const valarray<T>&) const;
        void operator|= (const valarray<T>&) const;
        void operator<<= (const valarray<T>&) const;
        void operator>>= (const valarray<T>&) const;
        ~mask_array();
    private:
        mask_array();
        mask_array(const mask_array<T>&);
        mask_array& operator=(const mask_array<T>&);
        ...
    };
}

```

Как и предыдущие классы, класс `gslice_array<>` предназначен исключительно для вспомогательных внутренних целей и должен быть доступен пользователю. Таким образом, все конструкторы и операция присваивания в классе `mask_array<>` являются закрытыми.

## Класс `indirect_array`

Объекты класса `indirect_array` создаются с помощью класса `valarray<size_t>`, используемого в качестве индекса неконстантного массива значений.

```

namespace std {
    template <typename T>
    class valarray {
    public:
        ...
        indirect_array<T> operator[] (const valarray<size_t>&);
        ...
    };
}

```

Класс `indirect_array` имеет следующие операции:

```
namespace std {
    template <typename T>
    class indirect_array {
    public:
        typedef T value_type;

        void operator= (const T&);
        void operator= (const valarray<T>&) const;
        void operator*= (const valarray<T>&) const;
        void operator/= (const valarray<T>&) const;
        void operator%= (const valarray<T>&) const;
        void operator+= (const valarray<T>&) const;
        void operator-= (const valarray<T>&) const;
        void operator^= (const valarray<T>&) const;
        void operator&= (const valarray<T>&) const;
        void operator|= (const valarray<T>&) const;
        void operator<= (const valarray<T>&) const;
        void operator>= (const valarray<T>&) const;
        ~indirect_array();
    private:
        indirect_array();
        indirect_array(const indirect_array<T>&);
        indirect_array& operator=(const indirect_array<T>&);
        ...
    };
}
```

Как и предыдущие классы, класс `indirect_array<>` предназначен исключительно для вспомогательных внутренних целей и должен быть доступен пользователю. Таким образом, все конструкторы и операция присваивания в классе `indirect_array<>` являются закрытыми.

## S.3. Подробное описание распределителей памяти и функций для работы с памятью

Как указано в разделе 4.6 и главе 19, распределители памяти представляют собой специальную модель памяти и являются абстракцией, используемой для перевода *потребности* в памяти в непосредственный *запрос* на выделение памяти. В разделе описываются детали использования распределителей памяти и управления памятью и, в частности, рассматриваются следующие темы.

- Распределители памяти с ограниченной областью видимости.
- Интерфейс распределителей памяти и распределитель памяти по умолчанию.
- Создание распределителей памяти до принятия стандарта C++11.
- Утилиты для неинициализированной памяти.

### S.3.1. Распределители памяти с ограниченной областью видимости

В стандарте C++98 концепция распределителей памяти сопровождалась рядом ограничений.

- Контейнерные объекты не обязаны были содержать свои распределители памяти в качестве членов. Распределители памяти были всего лишь частью типа. По этой причине имели место следующие ограничения.
- Распределители памяти, имеющие одинаковый тип, предполагались одинаковыми, так что память, выделенная одним распределителем, могла быть освобождена другим распределителем того же типа.
- Изменить ресурсы памяти в ходе выполнения программы было невозможно.
- Распределители памяти имели ряд ограничений на хранение состояния, например, в качестве регистрационной информации или информации о следующем участке выделяемой памяти.
- Распределители не обменивались при обмене контейнеров.
- Распределители памяти обрабатывались несогласованно, потому что копирующие конструкторы копировали распределителей, а операция присваивания нет. Таким образом, копирующий конструктор, с одной стороны, и конструктор по умолчанию в сочетании с операцией присваивания, с другой стороны, приводили к разным результатам.

Рассмотрим пример:

```
std::list<int> list1;    // эквивалент:
                        // std::list<int, std::allocator<int>> list1;
std::list<int, MyAlloc<int>> list2;

list1 == list2;         // ОШИБКА: разные типы
list1 = list2;          // ОШИБКА: разные типы
```

Это относится даже к строкам, которые являются разновидностью контейнеров STL.

```
typedef std::basic_string<char, std::char_traits<char>,
                        MyAlloc<char>> MyString;
std::list<std::string> list3;
std::list<MyString, MyAlloc<MyString>> list4;
...
list4.push_back(list3.front()); // ОШИБКА: разные типы элементов
std::equal_range(list3.begin(), list3.end(),
                 list4.begin(), list4.end()); // ОШИБКА: разные типы элементов
```

Для решения этих проблем в стандарте C++11 предложено несколько изменений.

- Компоненты библиотеки больше не должны считать, что все распределители памяти конкретного типа являются одинаковыми.
- *Свойства распределителей памяти* устраняют несколько проблем, создаваемых распределителями памяти, имеющими состояние.
- *Адаптер распределителей памяти с ограниченной областью видимости* предоставляет пользователю возможность передавать распределитель памяти от контейнера к содержащимся в нем элементам.



Эти усовершенствования образуют *инструментарий* для создания полиморфного распределителя памяти, позволяющего сравнивать или присваивать два контейнера с распределителями памяти разных типов, которые являются производными от общего типа распределителей памяти, используемого контейнерами. Однако из-за временных ограничений стандартный полиморфный распределитель памяти не вошел в стандарт C++11.<sup>6</sup>

Таким образом, предназначение класса `scoped_allocator_adaptor` заключается в предоставлении возможности передавать распределитель памяти от контейнера к содержащимся в нем элементам. Тем не менее для обеспечения обратной совместимости стандарт C++11 по-прежнему поддерживает старую модель распределителей памяти по умолчанию. Некоторые распределители памяти и свойства распределителей позволяют переключаться на новую модель, которую класс `scoped_allocator_adaptor` использует для обеспечения удобного интерфейса.

Для согласованного использования распределителя памяти контейнером и его элементами можно написать следующие выражения:

```
#include <scoped_allocator>

// используем стандартный распределитель памяти
// для контейнера и всех элементов
std::list<int, std::scoped_allocator_adaptor<std::allocator<int>>> list1;

// используем распределитель памяти MyAlloc<>
// для контейнера и всех элементов
std::list<int, std::scoped_allocator_adaptor<MyAlloc<int>>> list2;
```

Вместо этого можно использовать шаблонные синонимы (см. раздел 3.1.9). Рассмотрим пример:

```
#include <scoped_allocator>

template <typename T, typename Alloc>
using MyList = std::list<T, std::scoped_allocator_adaptor<Alloc<T>>>;

MyList<int, MyAlloc<int>>, list2;
```

Для использования разных распределителей памяти для элементов и контейнера необходимо передать распределитель памяти для элементов как дополнительный шаблонный параметр.

```
std::list<int, std::scoped_allocator_adaptor<MyAlloc1<int>, MyAlloc2<int>>> list;
```

Точно так же можно передать дополнительные типы распределителей памяти, чтобы задать распределитель памяти для элементов и т.д.

## S.3.2. Пользовательские распределители памяти в стандарте C++

Как было указано в разделе 19.2, можно просто создать собственный распределитель памяти, создав следующие компоненты.

- Определение типа для `value_type`, который передается как шаблонный параметр.
- Шаблонный конструктор, копирующий внутреннее состояние при изменении типа.

<sup>6</sup> Благодаря Пабло Гальперна (Pablo Halpern) за советы при написании этого раздела.

- Функция-член `allocate()`, выделяющая новую память.
- Функция-член `deallocate()`, освобождающая память, которая больше не нужна.
- Конструктор и деструктор, если они нужны, которые инициализируют, копируют и очищают внутреннее состояние.
- Операции `==` и `!=`.

Однако до принятия стандарта C++11 многие значения по умолчанию для пользовательских распределителей памяти не поддерживались. В результате приходилось создавать дополнительные члены, более или менее сложные. Рассмотрим аналог распределителя, описанного в разделе 19.2, который соответствует стандарту C++98:

```
// alloc/myalloc03.hpp

#include <cstddef>
#include <memory>
#include <limits>

template <typename T>
class MyAlloc {
public:
    // определения типов
    typedef std::size_t    size_type;
    typedef std::ptrdiff_t difference_type;
    typedef T*            pointer;
    typedef const T*       const_pointer;
    typedef T&             reference;
    typedef const T&        const_reference;
    typedef T              value_type;

    // конструкторы и деструктор
    // - ничего не делают, потому что распределитель
    // памяти не имеет состояния
    MyAlloc() throw() {}
    MyAlloc(const MyAlloc&) throw() {}

    template <typename U>
    MyAlloc (const MyAlloc<U>&) throw() {}
    ~MyAlloc() throw() {}

    // выделяем память, но не инициализируем num элементов типа T
    T* allocate (std::size_t num, const void* hint = 0) {
        // выделяем память глобальной операцией new
        return static_cast<T*> (::operator new(num*sizeof(T)));
    }

    // освобождаем область с адресом p, в которой записаны удаленные элементы
    void deallocate (T* p, std::size_t num) {
        // освобождаем область памяти с помощью глобальной операции delete
    }
};
```

```

        ::operator delete(p);
    }

    // возвращаем адрес значений
    T* address (T& value) const {
        return &value;
    }

    const T* address (const T& value) const {
        return &value;
    }

    // возвращаем максимальное количество элементов,
    // которые можно разместить в памяти
    std::size_t max_size () const throw() {
        // см. numeric_limits в разделе 5.3
        return std::numeric_limits<std::size_t>::max() / sizeof(T);
    }

    // инициализируем элементы выделенной памяти с адресом p
    // значением value
    void construct (T* p, const T& value) {
        // инициализируем память с помощью операции new с размещением
        ::new((void*)p)T(value);
    }

    // уничтожаем элементы, содержащиеся в инициализированной
    // области памяти с адресом p
    void destroy (T* p) {
        // уничтожаем объекты, вызывая их деструктор
        p->~T();
    }

    // связываем распределитель памяти с типом U
    template <typename U>
    struct rebind {
        typedef MyAlloc<U> other;
    };
};

// возвращаем признак того, что все специализации
// данного распределителя эквивалентны
template <typename T1, typename T2>
bool operator== (const MyAlloc<T1>&,
                 const MyAlloc<T2>&) throw() {
    return true;
}

template <typename T1, typename T2>
bool operator!= (const MyAlloc<T1>&,
                 const MyAlloc<T2>&) throw() {
    return false;
}

```

Как видим, нам пришлось создать несколько дополнительных типов, функции-члены `address()`, `max_size()`, `construct()`, `destroy()` и структуру `rebind`. Подробное описание этих членов приведено в разделе S.3.4.

## Шаблонный член `rebind<>`

Частью распределителя памяти является структура `rebind<>`. Эта шаблонная структура позволяет любому распределителю памяти выделять память для другого типа косвенным образом. Например, если `Allocator` — это тип распределителя, то `Allocator::rebind<T2>::other` — это распределитель, специализированный для элементов типа `T2`.

Структуру `rebind<>` можно использовать при реализации контейнера, в котором требуется выделить память для объекта, имеющего тип, отличающийся от типа элементов контейнера. Например, для реализации очереди обычно требуется память для массивов, управляющих блоками элементов (см. типичную реализацию класса `deque` в разделе 7.4). Таким образом, нам требуется распределитель памяти для выделения массивов указателей на элементы.

```
namespace std {
    template <typename T,
              typename Allocator = allocator<T> >
    class deque {
        ...
    private:
        // связываем распределитель памяти с типом T*
        typedef typename allocator_traits<Allocator>::rebind_alloc
            PtrAllocator;
        Allocator      alloc;           // распределитель памяти для значений типа T
        PtrAllocator    block_alloc;     // распределитель памяти для значений типа T*
        T**             elems;          // массив блоков элементов
        ...
    };
}
```

Для управления элементами очереди необходимо иметь один распределитель памяти для блоков элементов и другой для массива блоков элементов. Второй распределитель памяти имеет тип `PtrAllocator`, совпадающий с типом распределителя памяти для элементов. Используя структуру `rebind<>` распределитель памяти для элементов (`Allocator`) связывается с типом массива элементов (`T*`).

До принятия стандарта C++11 тип `PtrAllocator` должен был определяться следующим образом:

```
typedef typename Allocator:: template rebind<T*>::other
    PtrAllocator;
```

### S.3.3. Распределитель памяти по умолчанию

Распределитель памяти по умолчанию, который используется в тех случаях, когда среди шаблонных параметров не указан никакой конкретный распределитель памяти, объявлен следующим образом:<sup>7</sup>

<sup>7</sup>Благодарю Пабло Гальперна (Pablo Halpern) за советы при написании этого раздела.

```

namespace std {
    template <typename T>
    class allocator {
    public:
        // определения типов
        typedef size_t      size_type;
        typedef ptrdiff_t   difference_type;
        typedef T*          pointer;
        typedef const T*    const_pointer;
        typedef T&          reference;
        typedef const T&    const_reference;
        typedef T           value_type;

        // связываем распределитель памяти с типом U
        template <typename U>
        struct rebind {
            typedef allocator<U> other;
        };

        // возвращаем адрес значений
        pointer address(reference value) const noexcept;
        const_pointer address(const_reference value) const noexcept;

        // конструкторы и деструктор
        allocator() noexcept;
        allocator(const allocator&) noexcept;
        template <typename U>
        allocator(const allocator<U>&) noexcept;
        ~allocator();

        // возвращаем максимальное количество элементов,
        // которое можно разместить в памяти
        size_type max_size() const noexcept;

        // выделяем, но не инициализируем num элементов типа T
        pointer allocate(size_type num,
            allocator<void>::const_pointer hint = 0);

        // инициализируем элементы, размещенные в области с адресом p,
        // значениями из списка аргументов args
        template <typename U, typename... Args>
        void construct(U* p, Args&&... args);

        // удаляем элементы инициализированной области памяти с адресом p
        template <typename U>
        void destroy(U* p);

        // освобождаем область памяти с адресом p,
        // в которой были записаны удаленные элементы
        void deallocate(pointer p, size_type num);
    };
}

```

Для выделения и освобождения памяти распределитель памяти по умолчанию использует глобальные операции `new` и `delete`. Таким образом, функция-член `allocate()` может генерировать исключение `bad_alloc`. Однако распределитель памяти по умолчанию может быть оптимизирован с помощью повторного использования освобожденной памяти или выделения большего количества памяти, чем требуется, чтобы сэкономить время на дополнительных операциях выделения памяти. Итак, точные моменты времени, в которые выполняются операции `new` и `delete`, не определены. В разделе 19.2 приведена возможная реализация распределителя памяти по умолчанию.

Распределитель памяти по умолчанию имеет следующую специализацию для типа `void`:

```
namespace std {
    template <>
    class allocator<void> {
    public:
        typedef void* pointer;
        typedef const void* const_pointer;
        typedef void value_type;
        template <typename U>
        struct rebind {
            typedef allocator<U> other;
        };
    };
}
```

### S.3.4. Подробное описание распределителей

Для того чтобы выполнить указанные требования, распределители памяти должны иметь следующие типы и операции. Распределители, не предназначенные для стандартных контейнеров, могут иметь меньшее количество компонентов.

#### Определения типов

**`allocator::value_type`**

- Тип элементов.
- Обычно эквивалентен `T` для `allocator<T>`.

**`allocator::size_type`**

- Тип целочисленных значений без знака, представляющих размер самого большого объекта в модели выделения памяти.
- По умолчанию это тип `std::make_unsigned<allocator::difference_type>::type`, который обычно эквивалентен типу `std::size_t`.

**`allocator::difference_type`**

- Тип целочисленных значений со знаком, который представляет разность между двумя указателями в модели выделения памяти.
- По умолчанию это тип `std::pointer_traits<allocator::pointer>::difference_type`, который обычно эквивалентен типу `std::ptrdiff_t`.

***allocator* : pointer**

- Тип указателя на тип элемента.
- По умолчанию: `value_type*`.

***allocator* : const\_pointer**

- Тип указателя на тип элемента.
- По умолчанию: `const value_type*`.

***allocator* : void\_pointer**

- Тип указателя на тип `void`.
- По умолчанию: `void*`.
- Начиная со стандарта C++11.

***allocator* : const\_void\_pointer**

- Тип константного указателя на тип `void`.
- По умолчанию: `const void*`.
- Начиная со стандарта C++11.

***allocator* : reference**

- Тип ссылки на тип элемента.
- Обычно эквивалентен типу `T&` для `allocator<T>`.
- После принятия стандарта C++11 больше не требуется.

***allocator* : const\_reference**

- Тип константной ссылки на тип элемента.
- Обычно эквивалентен типу `const T&` для `allocator<T>`.
- После принятия стандарта C++11 больше не требуется.

***allocator* : rebind**

- Шаблонная структура, обеспечивающая любому распределителю памяти возможность косвенным образом выделять память для другого типа
- Должна быть объявлена следующим образом (это объявление по умолчанию принято в стандарте C++11).

```
template <typename T>
class allocator {
public:
    template <typename U>
    struct rebind {
        typedef allocator<U> other;
    };
    ...
}
```

- Объяснение предназначения структуры `rebind<>` приведено в разделе S.3.2.

***allocator::propagate\_on\_container\_copy\_assignment***

- Тип свойств для сигнализации о том, что распределитель памяти должен копироваться при выполнении операции присваивающего копирования контейнера.
- По умолчанию: `false_type`.
- Начиная со стандарта C++11.

***allocator::propagate\_on\_container\_move\_assignment***

- Тип свойств для сигнализации о том, что распределитель памяти должен перемещаться при выполнении операции перемещающего копирования контейнера.
- По умолчанию: `false_type`.
- Начиная со стандарта C++11.

***allocator::propagate\_on\_container\_swap***

- Тип свойств для сигнализации о том, что распределитель памяти должен обмениваться при обмене контейнеров.
- По умолчанию: `false_type`.
- Начиная со стандарта C++11.

## Операции

***allocator::allocator* ()**

- Конструктор по умолчанию
- Создает объект распределителя памяти.
- После принятия стандарта C++11 больше не требуется (однако хотя бы один не копирующий/неконвертирующий конструктор должен существовать).

***allocator::allocator* (const *allocator*& a)**

- Копирующий конструктор.
- Копирует объект распределителя памяти так, что область памяти, выделенная оригиналом или копией, может быть удалена одним из них.

***allocator::allocator* (*allocator*&& a)**

- Перемещающий конструктор.
- Если он не предусмотрен. используется копирующий конструктор.
- Перемещает объект распределителя памяти так, чтобы область, выделенная объектом a, могла быть освобождена объектом `*this`.
- Начиная со стандарта C++11.

***allocator::~allocator* ()**

- Деструктор.
- Уничтожает объект распределителя памяти.



pointer **allocator::address** (reference *value*)

const\_pointer **allocator::address** (const\_reference *value*)

- Первая версия возвращает неконстантный указатель на неконстантное значение *value*.
- Вторая форма возвращает константный указатель на константу *value*.
- После принятия стандарта C++11 больше не требуется.

size\_type **allocator::max\_size** ()

- Возвращает наибольшее разумное значение, которое можно передать функции-члену `allocate()` для выделения памяти.
- По умолчанию: `std::numeric_limits<allocator::size_type>::max()`.

pointer **allocator::allocate** (size\_type *num*)

pointer **allocator::allocate** (size\_type *num*,  
allocator<void>::const\_pointer *hint*)

- Обе версии возвращают память для *num* элементов типа *T*.
- Элементы не создаются/инициализируются (конструкторы не вызываются).
- Смысл необязательного второго элемента зависит от реализации. Например, его можно использовать для повышения производительности программы.
- Если *num* равно 0, то возвращаемое значение не определено.

void **allocator::deallocate** (pointer *p*, size\_type *num*)

- Освобождает память, на которую ссылается указатель *p*.
- Область памяти, на которую ссылается указатель *p*, должна быть выделена функцией-членом `allocate()` того же самого или эквивалентного распределителя памяти.
- Элементы должны быть предварительно уничтожены.

void **allocator::construct** (U\* *p*, Args&&... *args*)

- Инициализирует память для одного элемента, на который ссылается указатель *p*, списком аргументов *args*.
- По умолчанию: `::new((void*)p) U(std::forward<Args>(args) ...)`.
- До принятия стандарта второй аргумент имел тип `const T&`.

void **allocator::destroy** (U\* *p*)

- Уничтожает объект, на который ссылается указатель *p*, без освобождения памяти.
- Вызывает деструктор объекта.
- По умолчанию: `p->~U()`.

bool **operator ==** (const allocator& *a1*, const allocator& *a2*)

- Возвращает `true`, если распределители памяти *a1* и *a2* взаимозаменяемые.
- Два распределителя памяти являются взаимозаменяемыми, если область памяти, выделенная ими, может быть освобождена любым из них.

- До принятия стандарта C++11 при реализации стандартных контейнеров распределители памяти одного и того же типа должны были быть взаимозаменяемыми, поэтому эта функция всегда возвращала значение `true`.

`bool operator != (const allocator& a1, const allocator& a2)`

- Возвращает значение `true`, если два распределителя памяти не являются взаимозаменяемыми.
- Эквивалент `!(a1 == a2)`.
- До принятия стандарта C++11 при реализации стандартных контейнеров распределители памяти одного и того же типа должны были быть взаимозаменяемыми, поэтому эта функция всегда возвращала значение `false`.

`allocator select_on_container_copy_construction ()`

- Возвращает распределитель памяти, допускающий копирование при создании копии контейнера.
- По умолчанию возвращает `*this`.
- Начиная со стандарта C++11.

### S.3.5. Подробное описание утилит для работы с неинициализированной памятью

В разделе описываются вспомогательные функции для работы с неинициализированной памятью. Реализация этих функций, безопасная с точки зрения исключений, публикуется с разрешения Грегга Колвина (Greg Colvin).

`void uninitialized_fill (ForwardIterator beg, ForwardIterator end, const T& value)`

- Инициализирует элементы в интервале `[beg, end)` значением `value`.
- Функция либо выполняется успешно, либо не выполняется вообще.
- Функция обычно реализуется следующим образом:

```
namespace std {
    template <typename ForwIter, typename T>
    void uninitialized_fill(ForwIter beg, ForwIter end,
                           const T& value)
    {
        typedef typename iterator_traits<ForwIter>::value_type VT;
        ForwIter save(beg);
        try {
            for (; beg!=end; ++beg) {
                ::new (static_cast<void*>(&*beg)) VT(value);
            }
        }
        catch (...) {
            for (; save!=beg; ++save) {
                save->~VT();
            }
        }
    }
}
```

```

        throw;
    }
}

```

ForwardIterator **uninitialized\_fill\_n** (ForwardIterator *beg*,  
Size *num*, const T& *value*)

- Инициализирует *num* элементов, начиная с позиции *beg*, значением *value*.
- Возвращает позицию, следующую за последним инициализированным элементом.
- Функция либо выполняется успешно, либо не выполняется вообще.
- Функция обычно реализуется следующим образом:

```

namespace std {
    template <typename ForwIter, typename Size, typename T>
    ForwIter uninitialized_fill_n (ForwIter beg, Size num,
                                  const T& value)
    {
        typedef typename iterator_traits<ForwIter>::value_type VT;
        ForwIter save(beg);
        try {
            for ( ; num--; ++beg) {
                ::new (static_cast<void*>(&*beg)) VT(value);
            }
            return beg;
        }
        catch (...) {
            for ( ; save!=beg; ++save) {
                save->~VT();
            }
            throw;
        }
    }
}

```

- Пример использования функции `uninitialized_fill_n()` приведен в разделе 19.3.
- До принятия стандарта C++11 возвращала значение типа `void`.

ForwardIterator **uninitialized\_copy** (InputIterator *sourceBeg*,  
InputIterator *sourceEnd*,  
ForwardIterator *destBeg*)

- Инициализирует память, начиная с позиции *destBeg*, элементами из интервала [*sourceBeg*,*sourceEnd*).
- Возвращает позицию, следующую за последним инициализированным элементом.
- Функция либо выполняется успешно, либо не выполняется вообще.
- Функция обычно реализуется следующим образом:

```

namespace std {
    template <typename InputIter, typename ForwIter>

```

```

ForwardIterator uninitialized_copy(InputIterator pos, InputIterator end,
                                ForwardIterator dest)
{
    typedef typename iterator_traits<ForwardIterator>::value_type VT;
    ForwardIterator save(dest);
    try {
        for (; pos!=end; ++pos, ++dest) {
            ::new (static_cast<void*>(&*dest)) VT(*pos);
        }
        return dest;
    }
    catch (...) {
        for (; save!=dest; ++save) {
            save->~VT();
        }
        throw;
    }
}

```

- Пример использования функции `uninitialized_copy()` приведен в разделе 19.3.

ForwardIterator **uninitialized\_copy\_n** (InputIterator *sourceBeg*,  
Size *num*, ForwardIterator *destBeg*)

- Инициализирует *num* элементов памяти, начиная с позиции *destBeg*, элементами, начиная с позиции *sourceBeg*.
- Возвращает позицию, следующую за последним инициализированным элементом.
- Функция либо выполняется успешно, либо не выполняется вообще.
- Функция обычно реализуется следующим образом:

```

namespace std {
    template <typename ForwardIterator, typename Size, typename ForwardIterator>
    ForwardIterator uninitialized_copy_n(InputIterator pos, Size num,
                                       ForwardIterator dest)
    {
        typedef typename iterator_traits<ForwardIterator>::value_type VT;
        ForwardIterator save(dest);
        try {
            for ( ; num>0; ++pos, ++dest, --num) {
                ::new (static_cast<void*>(&*dest)) VT(*pos);
            }
            return dest;
        }
        catch (...) {
            for (; save!=dest; ++save) {
                save->~VT();
            }
            throw;
        }
    }
}

```

- Начиная со стандарта C++11.